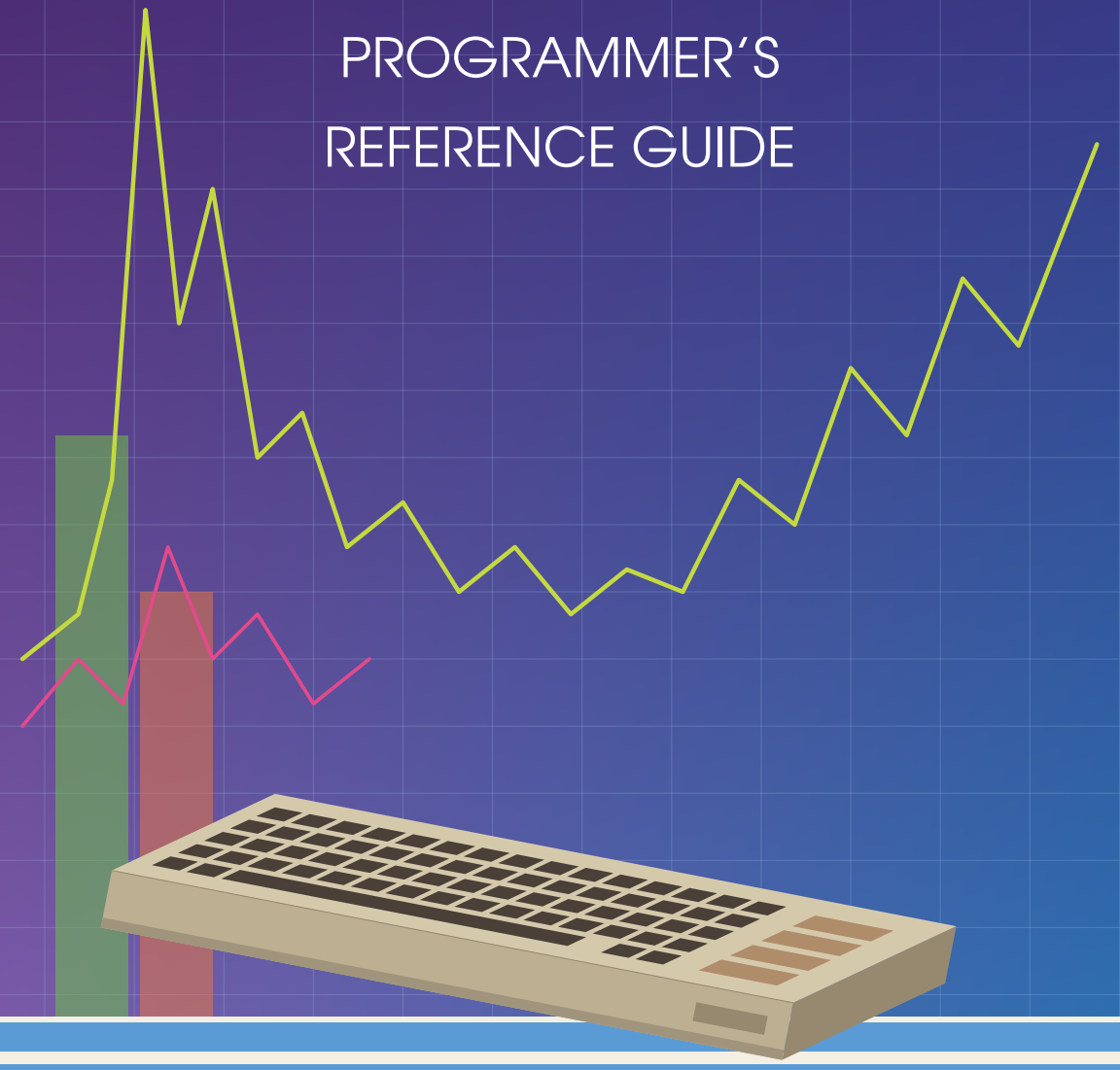
ULTIMATE SDK

PROGRAMMER'S REFERENCE GUIDE



ULTIMATE SDK

PROGRAMMER'S REFERENCE GUIDE

For Commodore 64 programs using the Ultimate Command Interface

First edition.

This guide documents the Ultimate SDK, a software development kit for the Commodore 64 and Ultimate hardware with the command interface.

Released under the MIT licence. Commodore, the Commodore logo and Commodore 64 are trademarks of their respective owners; this guide is not affiliated with or endorsed by them.

Contents

Introduction v

1 GETTING STARTED 2
What The SDK Requires . . . 2
Enable The Command Interface . . . 2
Choose A Binding . . . 2
Recover From Common Failures . . . 4

2 PROGRAMMING IN BASIC 7
Two Ways To Install The Same Wedge . . . 7
Every BASIC Keyword By Example . . . 7
BASIC-specific Limits . . . 10

3 PROGRAMMING IN C 13
How To Read The Examples . . . 13
Start And Discover The Machine . . . 14
Read And Change The Live Palette . . . 15
Use Ultimate 64 Turbo Control . . . 16
Change And Read The Current Directory . . . 16
Open, Read And Close A File . . . 18
Load And Save C64 Memory . . . 19
Move Bytes Through The REU . . . 20
Inspect Network Interfaces . . . 22
Use TCP And UDP Sockets . . . 23
Fetch A URL In One Call . . . 24
Build A Multi-step HTTP Request . . . 24
Probe And Configure The Raw Transport . . . 26
Execute A Raw Command With And Without Arguments . . . 27
Read A Raw Multi-block Reply . . . 28
Decode And Inspect Raw Status . . . 29

4 PROGRAMMING IN ASSEMBLY 31
The Calling Convention . . . 31

Place The SDK's Working Memory 32
Initialize, Detect And Describe The Machine 32
Read And Change The Palette 35
Use Turbo And Badline Control 36
Change And Read The Current Directory 37
Open, Seek, Read, Write And Close Files 39
Load And Save C64 Memory 41
Move Bytes Through The REU 42
Inspect Network Interfaces 45
Use TCP And UDP Sockets 46
Use The HTTP Client 48
Probe And Configure The Raw Transport 51
Raw Commands: Optional Arguments 52
Read A Raw Multi-block Reply 54

5 OTHER TOOLCHAINS **56**
Build And Identify The Blob 56
Call The Stable Jump Table 56
Use The Parameter Block 57
Call The Blob From BASIC 57
Choose A Different Base Address 57

6 APPENDIX **59**
Result Codes 59

INTRODUCTION

The **ULTIMATE SDK** gives a C64 program one API for the Ultimate command interface and for the two supported features that use direct hardware registers: REU transfers and Ultimate 64 turbo control. The language chapters cover every public call.

The implementation is 6502 assembly. cc65 links it as a library; ca65 callers use its assembly ABI; BASIC reaches a banked build through a wedge; other toolchains call the same object code through a standalone jump-table binary.

How to read this

Chapter 1 covers setup and chooses a binding. Chapters 2 to 5 document the binding-specific rules and every public entry point. Each example says what feature it demonstrates; the appendix holds the result-code table.

A word about the name

The library is called the Ultimate SDK rather than the Ultimate 64 SDK because the command interface predates the Ultimate 64. Programs probe targets with `ultimate_detect()` and test the REU and turbo registers separately.

CHAPTER 1

GETTING STARTED

- What the SDK requires
- Enabling the command interface
- Choosing and checking a binding
- Recovering from common failures

WHAT THE SDK REQUIRES

The SDK runs on a C64 with Ultimate hardware whose command interface is mapped at `$DF1B–$DF1F`. It is supported on the 1541 Ultimate-II and later products.

There is no blanket minimum firmware version. Targets were added over time, so programs call `ultimate_detect()` and test the capability they need. The palette is newer than firmware 3.15; HTTP appears in 3.15.

The control target predates the palette commands, so its presence is not a palette capability test. Call a palette function and handle `ULTIMATE_ERR_NOT_SUPPORTED`; the SDK returns that result when older firmware answers that it does not know the command.

ENABLE THE COMMAND INTERFACE

Open the Ultimate configuration menu and set `COMMAND INTERFACE` to `ENABLED`. The exact key used to open that menu depends on the product and its configuration, so the SDK does not prescribe one.

With the interface disabled, the five registers are not mapped and `ultimate_init()` returns `ULTIMATE_ERR_NO_DEVICE`.

REU transfers and Ultimate 64 turbo control use their own registers, not the command interface. Each still depends on its corresponding machine setting.

CHOOSE A BINDING

BASIC V2	`make wedge`; load `src/basic/uci.prg`, or mount `src/basic/uci.crt`
cc65 C	`make lib`; include `ultimate.h` and link `bindings/cc65/build/ultimate.lib`
ca65 assembly	link the same library and include `bindings/asm/ultimate.inc`
other toolchains	`make blob`; load the standalone binary described in Chapter 5

BASIC check

Load and run the wedge, list the current directory, then print the command result:

```
LOAD"UCI",8
RUN
UDIR
PRINT UERR
0
```

This demonstrates wedge installation, a DOS command and the shared result model. A printed `0` is `ULTIMATE_OK`; `1` is `ULTIMATE_ERR_NO_DEVICE`.

C check

This complete program demonstrates initialization and error reporting:

```c
#include <stdio.h>
#include <ultimate.h>

int main(void)
{
  uint8_t err = ultimate_init();

  if (err != ULTIMATE_OK)
    printf("ultimate: %s\n", ultimate_strerror(err));
  return err != ULTIMATE_OK;
}
```

OUTPUT

```
No text on success. On failure, for example:
ULTIMATE: NO ULTIMATE FOUND
```

Build it from the repository root:

```
cl65 -t c64 -I include -o hello.prg hello.c \
    bindings/cc65/build/ultimate.lib
```

OUTPUT

```
hello.prg is created with no compiler errors.
```

Assembly check

This demonstrates the assembly return convention: the result code is in `A`, ready to compare and branch on.

```
    jsr ultimate_init
    cmp #ULTIMATE_OK
    bne failed
```

OUTPUT

```
A = ULTIMATE_OK; BNE is not taken.
```

The `cmp` instruction leaves the processor flags ready for `bne`.

RECOVER FROM COMMON FAILURES

No device

`ULTIMATE_ERR_NO_DEVICE` means the signature is absent. Check the Command Interface setting before assuming the hardware is missing.

An abandoned exchange

A program can stop while the firmware is holding command or reply state. `uci_abort()` releases it; `ultimate_init()` performs that recovery during startup. From BASIC, a bare `UCI` calls the same abort path:

```
UCI:PRINT UERR
0
READY.
```

This example demonstrates recovery only; it does not send a command.

A missing feature

Use `ultimate_detect()` for command-interface targets, `ultimate_reu_available()` for the REU, and `ultimate_turbo_available()` for turbo. Do not infer capabilities from a model name.

A short buffer

`ULTIMATE_ERR_TRUNCATED` means an exchange completed but not all reply bytes fitted. Output lengths report the bytes retained, not the bytes discarded.

CHAPTER 2

PROGRAMMING IN BASIC

- How the two wedge builds differ
- The twenty-three keywords
- Clear examples of each service group
- BASIC-specific limits

TWO WAYS TO INSTALL THE SAME WEDGE

Chapter 1 shows the `uci.prg` installation. Its loader copies the SDK under BASIC ROM at `$A000` and the wedge to RAM at `$C000`, installs four BASIC vectors, then performs `NEW`. The normal BASIC program area remains available.

The `uci.crt` build carries the same runtime bytes in an 8K cartridge. It autostarts and its warm-start vector reinstalls the wedge after reset. While the cartridge remains mapped at `$8000–$9FFF`, BASIC's program area is 8K smaller.

Both builds contain the same wedge; only their loaders differ.

> **Careful:** The wedge owns BASIC's tokenise, list, statement-dispatch and expression vectors and does not chain previous owners. Do not combine it with another extension that needs any of those vectors unless that combination provides an explicit compatibility layer.

EVERY BASIC KEYWORD BY EXAMPLE

The wedge exposes twenty-three keywords. This section uses every one. Services without a dedicated keyword remain available through the raw `UCI` statement. Output below is from a representative successful run; model names, times, directory entries and REU size depend on the machine. When a result is fixed, it appears directly after `RUN` in the same screen.

Run a command and inspect every result

```
10 UCI UCTRL,1
20 PRINT "RESULT";UERR;" DEVICE";UDEV;" BYTES";ULEN
30 PRINT "STATUS: ";UST$
40 PRINT "DATA: ";UDAT$
50 FOR I=0 TO ULEN-1:PRINT UBYTE(I);" ";:NEXT:PRINT
RUN
RESULT 0 DEVICE 0 BYTES 14
STATUS: 00,OK
DATA: CONTROL TARGET
67 79 78 84 82 79 76 32 84 65 82 71 69 84
READY.
```

`UCI UCTRL,1` asks the control target to identify itself. `UERR` is the SDK result, `UDEV` is the target's raw status number, and `ULEN` is the retained reply length. `UST$` returns the status text, `UDAT$` returns up to 255 reply bytes as a string, and `UBYTE(N)` returns one retained

byte or zero when `N` is past the end.

Use every target constant

```
10 PRINT UDOS1,UDOS2,UNET,UCTRL,UIEC,UHTTP
RUN
1  2  3  4  5  6
READY.
```

The printed values are 1 through 6: the two DOS instances, network, control, SoftwareIEC and HTTP targets. Use these names as the first argument to `UCI`; do not copy their numeric values into programs.

Send a raw command with and without an optional argument

```
10 UCI UDOS1,38: PRINT UDAT$
20 UCI UDOS1,38,0: PRINT UDAT$
30 UCI UCTRL,40: PRINT UDAT$
40 UCI UCTRL,40,0: PRINT UDAT$
50 IF UERR THEN PRINT "COMMAND FAILED";UDEV
```

Command 38 is `DOS_CMD_GET_TIME`. Line 10 omits its optional format byte; line 20 supplies it explicitly. Command 40 is `CTRL_CMD_GET_HWINFO`; lines 30 and 40 show its omitted and explicit model selector. Both commands default an omitted argument to zero. A number normally contributes one byte; known wide arguments are widened automatically.

Force raw argument widths

```
10 UCI UDOS1,240,UW(4660),UL(65536)
20 FOR I=0 TO ULEN-1:PRINT UBYTE(I);" ";:NEXT:PRINT
RUN
1 240 52 18 0 0 1 0
READY.
```

Command 240 echoes the target byte, command byte and arguments. The leading 1 and 240 confirm the framing. `UW(4660)` sends 52,18 and `UL(65536)` sends 0,0,1,0: both are

little-endian. These overrides are for commands newer than the wedge's shape table or deliberately raw byte layouts.

Abort an unfinished exchange

```
UCI:PRINT UERR
0
READY.
```

A bare `UCI` sends no command. It aborts any exchange left unfinished by a stopped program and returns the interface to idle; `UERR` reports whether the recovery succeeded.

Set and read turbo

```
10 UTURBO 3: IF UERR THEN PRINT "TURBO UNAVAILABLE":END
20 PRINT "SPEED INDEX";UTURBO
RUN
SPEED INDEX 3
READY.
```

The statement form sets an index; the function form reads it. Portable indices 0 through 3 select 1, 2, 3 and 4MHz. A read of 255 means the turbo registers are unavailable.

Load a PRG with and without the optional address

```
10 ULOAD "GAME.PRG": IF UERR THEN END
20 ULOAD "FONT.PRG",49152: IF UERR THEN END
```

Line 10 consumes the PRG's two-byte header and loads at the address stored there. Line 20 still consumes the header but loads the remaining bytes at 49152. Dedicated filename keywords copy at most 63 bytes and add a terminator.

Load and save raw memory

```
10 UBLOAD "FONT.BIN",49152,2048: IF UERR THEN END
20 USAVE "SCREEN.BIN",1024,1000: IF UERR THEN END
```

`UBLOAD` reads at most 2048 raw bytes into address 49152; it consumes no header. `USAVE` replaces `SCREEN.BIN` with 1000 bytes beginning at C64 address 1024.

List the current directory

```
UDIR
```

`UDIR` takes no parameters. It opens the current Ultimate directory, prints every entry and leaves `UERR` at zero when the normal end marker is reached.

Copy memory to and from the REU

```
10 R=UREU:PRINT "REU BANKS";R:IF R=0 THEN END
20 USTASH 1024,0,1000:IF UERR THEN END
30 PRINT CHR$(147)
40 UFETCH 1024,0,1000:IF UERR THEN END
```

`UREU` returns the expansion size in 64K banks. `USTASH` copies the 1000-byte C64 text screen to REU address zero; `UFETCH` copies it back. The REU address is 24 bits. Keep the address plus length within `UREU` times 65536; a DMA length of zero means 65536 bytes.

BASIC-SPECIFIC LIMITS

A statement extension cannot follow THEN directly

BASIC V2's `IF` path enters the ROM statement dispatcher instead of the wedge vector, so this is rejected:

```
10 IF X = 1 THEN ULOAD "GAME.PRG"
RUN
?SYNTAX ERROR IN 10
READY.
```

Branch to a line containing the extension statement:

```
10 IF X = 1 THEN 100
20 END
100 ULOAD "GAME.PRG"
```

This demonstrates the workaround. Extension functions are unaffected, so `IF UERR THEN 100` remains valid.

Reply storage is finite

The wedge retains 256 reply bytes. `UDAT$` can expose only 255 because that is BASIC's maximum string length; `UBYTE(0)` through `UBYTE(255)` can inspect the retained bytes individually. A longer raw reply sets `UERR` to `ULTIMATE_ERR_TRUNCATED`.

CHAPTER 3

PROGRAMMING IN C

- Startup, discovery and diagnostics
- Machine, file and REU services
- Network and HTTP services
- The complete low-level UCI API

HOW TO READ THE EXAMPLES

Every public function from `ultimate.h` and `uci.h` appears below. Each listing shows one complete operation within a program; calls share state only when the text says so. The guide build compiles every displayed call form. Unless a listing shows otherwise, it runs after including `stdio.h`, `string.h`, `ultimate.h` and `uci.h`. Result-returning calls use this pattern:

```
uint8_t err = ultimate_init();
if (err != ULTIMATE_OK)
  printf("init: %s\n", ultimate_strerror(err));
```

OUTPUT

```
No text on success. On failure, for example:
INIT: TIMEOUT
```

Output boxes show a representative successful run on an Ultimate 64 Elite. Model names, paths, directory entries, addresses and network replies naturally vary with the machine and the files or servers used.

cc65 maps lowercase source letters to the uppercase byte values shared by PETSCII and ASCII. For that reason, path, filename and header literals appear in lowercase source even though the firmware receives uppercase text.

Zero is success. Appendix A lists the remaining results. Include `ultimate.h` for the service API; include `uci.h` only for the raw transport examples at the end of this chapter.

START AND DISCOVER THE MACHINE

```
int main(void)
{
  ultimate_capabilities caps;
  char text[64];
  uint16_t length;
  uint8_t err = ultimate_init();

  if (err != ULTIMATE_OK || !ultimate_available()) return 1;
  err = ultimate_detect(&caps);
  if (err == ULTIMATE_OK && ultimate_has_http(&caps))
    puts("http target present");

  err = ultimate_identify(UCI_TARGET_DOS1,
      text, sizeof text, &length);
  if (err == ULTIMATE_OK) printf("dos: %s\n", text);
  err = ultimate_identify(UCI_TARGET_DOS1,
      text, sizeof text, NULL);

  err = ultimate_get_model(text, sizeof text, &length);
  if (err == ULTIMATE_OK)
    printf("model received: %u bytes\n", length);
  err = ultimate_get_model(text, sizeof text, NULL);
  return err != ULTIMATE_OK;
}
```

OUTPUT

```
HTTP TARGET PRESENT
DOS: ULTIMATE-II DOS V1.2
MODEL RECEIVED: 17 BYTES
```

`ultimate_init()` performs the required startup check; `ultimate_available()` reads the resulting SDK state without touching hardware. `ultimate_detect()` fills `caps`; use the `ultimate_has_*()` macros to test targets. The paired identify and model calls show the optional final parameter: pass `&length` to receive the stored string length, or `NULL` when only the terminated string matters.

Report an error and its raw device code

```
if (err != ULTIMATE_OK) {
  printf("%s (device %u)\n",
    ultimate_strerror(err), ultimate_device_code());
}
```

OUTPUT

```
TIMEOUT (DEVICE 0)
```

`ultimate_strerror()` returns stable SDK text. `ultimate_device_code()` returns the last target-specific numeric status for diagnostics; do not use that firmware-specific number as application control flow.

READ AND CHANGE THE LIVE PALETTE

```
uint8_t palette[ULTIMATE_PALETTE_BYTES];
uint8_t err;

err = ultimate_palette_get(palette);
if (err == ULTIMATE_OK) {
  palette[0] = 0;
  palette[1] = 0;
  palette[2] = 0;
  err = ultimate_palette_set(palette);
}

err = ultimate_palette_set_color(6, 255, 0, 0);
err = ultimate_palette_reset();
```

OUTPUT

```
SET COLOUR 6 TO RED: ULTIMATE_OK
RESTORE BUILT-IN PALETTE: ULTIMATE_OK
```

`ultimate_palette_get()` reads all sixteen RGB triples and `ultimate_palette_set()` writes all of them. The single-colour call sets index 6 to red. `ultimate_palette_reset()` restores the built-in palette; none of these calls writes flash or a palette file. On firmware without the palette commands, each call returns `ULTIMATE_ERR_NOT_SUPPORTED`; test the

result before using data from `ultimate_palette_get()`.

USE ULTIMATE 64 TURBO CONTROL

```
uint8_t speed;
uint8_t err;

if (ultimate_turbo_available()) {
  speed = ultimate_turbo_get();
  err = ultimate_turbo_set(U64_SPEED_4MHZ);
  err = ultimate_turbo_badlines(0);
  err = ultimate_turbo_badlines(1);
  err = ultimate_turbo_set(speed);
}
```

OUTPUT

```
AVAILABLE = 1
PREVIOUS SPEED = U64_SPEED_1MHZ
SET 4 MHZ, THEN RESTORE SPEED AND BADLINES: ULTIMATE_OK
```

`ultimate_turbo_available()` is the capability check and `ultimate_turbo_get()` reads the index. The set call changes only speed. The two badline calls explicitly show both forms: zero disables badlines and non-zero restores them. The final set restores the speed read at entry.

CHANGE AND READ THE CURRENT DIRECTORY

```
char path[128];
uint16_t length;
uint8_t err;

err = ultimate_chdir("/usb1/games");
err = ultimate_getpath(path, sizeof path, &length);
err = ultimate_getpath(path, sizeof path, NULL);
```

OUTPUT

```
path = "/USB1/GAMES"
length = 11
```

The change-directory call selects an absolute or relative firmware path. The paired getpath calls show its optional output: receive the stored length through `&length`, or pass `NULL` when the terminated path is enough.

Read every directory entry

```
char name[64];
uint8_t attrib;
uint8_t err = ultimate_opendir();

while (err == ULTIMATE_OK) {
  err = ultimate_readdir(name, sizeof name, &attrib);
  if (err == ULTIMATE_OK) puts(name);
}
if (err == ULTIMATE_END) err = ULTIMATE_OK;
```

OUTPUT

```
DATA
BIG.BIN
README.TXT
```

`ultimate_opendir()` starts one live exchange and each `ultimate_readdir()` releases one entry. `ULTIMATE_END` is normal. Issue no unrelated call inside the loop.

When attributes are not needed, pass the documented optional value:

```
err = ultimate_readdir(name, sizeof name, NULL);
```

OUTPUT

```
name receives the entry; no attribute byte is stored.
```

If a program stops the walk before `ULTIMATE_END`, call `uci_abort()` before issuing another command.

OPEN, READ AND CLOSE A FILE

```
uint8_t buf[128];
uint16_t got;
uint8_t err;
uint8_t close_err;

err = ultimate_open("readme.txt", DOS_FA_READ);
if (err == ULTIMATE_OK) {
  err = ultimate_seek(0);
  if (err == ULTIMATE_OK)
    err = ultimate_read(buf, sizeof buf, &got);
  close_err = ultimate_close();
  if (err == ULTIMATE_OK) err = close_err;
}
```

OUTPUT

```
got = 128
err = ULTIMATE_OK
```

`ultimate_open()` establishes the firmware's current file. `ultimate_seek()` selects an absolute position, `ultimate_read()` reads from it, and `ultimate_close()` releases the same file. A short read is end-of-file, not an error; `got` is the number stored.

The byte-count output is optional:

```
err = ultimate_read(buf, sizeof buf, NULL);
```

OUTPUT

```
buf receives bytes and the file advances; the count is discarded.
```

That form still advances the open file but discards the count.

Create, write and delete a file

```
static const uint8_t message[] = {0x4f,0x4b,13};

err = ultimate_open("result.txt",
    DOS_FA_CREATE_ALWAYS | DOS_FA_WRITE);
if (err == ULTIMATE_OK) {
  err = ultimate_write(message, sizeof message);
  close_err = ultimate_close();
  if (err == ULTIMATE_OK) err = close_err;
}
err = ultimate_delete("result.txt");
```

OUTPUT

```
RESULT.TXT: WROTE 3 BYTES, CLOSED, DELETED
```

The open flags create or replace the firmware-side file. `ultimate_write()` appends the specified bytes at its current position; `ultimate_delete()` removes the named file after it is closed.

LOAD AND SAVE C64 MEMORY

```
uint16_t end;

err = ultimate_load("game.prg", 0);
end = ultimate_last_end();

err = ultimate_load("font.prg", 0xC000);
end = ultimate_last_end();

err = ultimate_bload("font.bin", 0xC000, 2048);
err = ultimate_save("screen.bin", 0x0400, 1000);
```

OUTPUT

```
GAME.PRG END = address after loaded data
FONT.BIN: 2048 bytes loaded at $C000
SCREEN.BIN: 1000 bytes saved
```

The first `ultimate_load()` uses the PRG header address; the second shows the explicit-address form. Both consume the header, and `ultimate_last_end()` returns the address following the last byte. `ultimate_bload()` reads bounded raw bytes without a header. `ultimate_save()` replaces a file with the named C64 memory range.

MOVE BYTES THROUGH THE REU

```
uint16_t banks;

if (ultimate_reu_available()) {
  banks = ultimate_reu_size();
  err = ultimate_reu_stash(0x0400, 0, 1000);
  err = ultimate_reu_fetch(0x0400, 0, 1000);
}
```

OUTPUT

```
banks = installed REU size in 64K banks
1000 bytes stashed and fetched: ULTIMATE_OK
```

Availability is separate from size, which is returned in 64K banks. Stash copies C64 RAM to the REU; fetch copies the same range back. A DMA length of zero means 65536 bytes.

Move the current file directly to and from the REU

File to REU

```
err = ultimate_open("assets.bin", DOS_FA_READ);
if (err == ULTIMATE_OK) {
  err = ultimate_seek(0);
  if (err == ULTIMATE_OK)
    err = ultimate_reu_load(0, 65536UL);
  ultimate_close();
}
```

OUTPUT

```
ASSETS.BIN TO REU $000000: 65536 BYTES
```

REU to file

```
err = ultimate_open("capture.bin",
    DOS_FA_CREATE_ALWAYS | DOS_FA_WRITE);
if (err == ULTIMATE_OK) {
  err = ultimate_reu_save(0, 65536UL);
  ultimate_close();
}
```

OUTPUT

```
REU $000000 TO CAPTURE.BIN: 65536 BYTES
```

These functions take no filename or file handle. The first flow opens the firmware's current DOS file, selects its first byte, then copies it to REU address zero. The second flow makes a different current file and copies REU bytes into it. Close the same firmware-side file after either transfer.

INSPECT NETWORK INTERFACES

```
uint8_t count;
uint8_t mac[6];
uint8_t ipconfig[12];

err = ultimate_net_ifcount(&count);
if (err == ULTIMATE_OK && count != 0) {
  err = ultimate_net_macaddr(0, mac);
  err = ultimate_net_ipconfig(0, ipconfig);
}
```

OUTPUT

```
count = number of network interfaces
mac = six bytes; ipconfig = twelve bytes
```

The count call reports available interfaces. The next two calls read the six-byte MAC address and twelve-byte IPv4 configuration for interface zero into caller-owned buffers.

USE TCP AND UDP SOCKETS

```
uint8_t handle;
uint8_t buf[128];
uint16_t got;
uint16_t sent;
static const uint8_t message[] = {0x48,0x49,13};

err = ultimate_net_connect("192.168.1.50", 6502, &handle);
if (err == ULTIMATE_OK) {
  err = ultimate_net_write(handle, message,
      sizeof message, &sent);
  do {
    err = ultimate_net_read(handle, buf, sizeof buf, &got);
  } while (err == ULTIMATE_OK && got == 0);
  if (err != ULTIMATE_END)
    ultimate_net_close(handle);
}

err = ultimate_net_udp("192.168.1.51", 6502, &handle);
if (err == ULTIMATE_OK)
  err = ultimate_net_close(handle);
```

OUTPUT

```
TCP: 3 BYTES SENT; REPLY BYTES RETURNED IN got
UDP HANDLE OPENED AND CLOSED
```

`ultimate_net_connect()` opens TCP; `ultimate_net_udp()` opens UDP. Both return a handle. Write reports bytes accepted through `sent`; read reports bytes stored through `got`. A successful zero-byte read means “not yet”, so poll. `ULTIMATE_END` means the peer closed and the handle is already released; call `ultimate_net_close()` only for a handle still owned by the program.

FETCH A URL IN ONE CALL

```
static const char url[] = {
  0x68,0x74,0x74,0x70,0x3a,0x2f,0x2f,
  0x31,0x39,0x32,0x2e,0x31,0x36,0x38,0x2e,0x31,
  0x2e,0x35,0x30,0x2f,0
};
uint8_t reply[512];
uint16_t got;

err = ultimate_http_get(url, reply, sizeof reply, &got);
```

OUTPUT

```
err = ULTIMATE_OK
got = HTTP response-body bytes stored in reply
```

`ultimate_http_get()` creates, exchanges and closes one GET request. The explicit bytes prevent cc65 from changing lowercase URL characters to the C64 character set. `got` is the body bytes retained; a larger body returns `ULTIMATE_ERR_TRUNCATED` and has no continuation.

BUILD A MULTI-STEP HTTP REQUEST

This example reuses `url`, `reply` and `got` from the preceding one.

```
uint8_t request;
uint8_t close_err;
uint8_t body_handle; /* returned by raw HTTP body commands */

err = ultimate_http_open(HTTP_VERB_GET, url, &request);
if (err == ULTIMATE_OK) {
  err = ultimate_http_header(request, "accept: text/plain");
  if (err == ULTIMATE_OK)
  err = ultimate_http_exchange(request, HTTP_BODY_NONE,
      reply, sizeof reply, &got);
  close_err = ultimate_http_close(request);
  if (err == ULTIMATE_OK) err = close_err;
}

err = ultimate_http_open(HTTP_VERB_POST, url, &request);
if (err == ULTIMATE_OK) {
  err = ultimate_http_exchange(request, body_handle,
      reply, sizeof reply, &got);
  close_err = ultimate_http_close(request);
  if (err == ULTIMATE_OK) err = close_err;
}

err = ultimate_http_free_all();
```

OUTPUT

```
GET WITHOUT BODY: RESPONSE STORED
POST WITH BODY HANDLE: RESPONSE STORED
ALL HTTP OBJECTS FREED
```

Open returns the request handle used by header, exchange and close. The first exchange explicitly passes `HTTP_BODY_NONE`; the second shows the optional body-handle form. Body construction itself uses raw UCI commands. Always close a handle you own. `ultimate_http_free_all()` is the recovery call for slots abandoned by an earlier program.

PROBE AND CONFIGURE THE RAW TRANSPORT

```
uint8_t present = uci_signature_present();
uint8_t ident = uci_ident_register();
uint8_t old_timeout;

err = uci_init();
old_timeout = uci_get_timeout();
uci_set_timeout(UCI_TIMEOUT_FOREVER);
/* run the deliberately unbounded command */
uci_set_timeout(old_timeout);
err = uci_abort();
```

OUTPUT

```
present = 1
ident = command-interface identification byte
abort = ULTIMATE_OK
```

The signature and identification-register calls are immediate register reads. `uci_init()` performs the functional startup check. Timeout zero means wait forever; save and restore the old value. `uci_abort()` releases any exchange still in progress and is safe when the interface is already idle.

EXECUTE A RAW COMMAND WITH AND WITHOUT ARGUMENTS

```
uci_request req = {0};
uint8_t reply[64];
uint8_t status[16];
uint8_t format = 0;
uint8_t sub = CTRL_HWINFO_MODEL;

req.target = UCI_TARGET_DOS1;
req.command = UCI_CMD_IDENTIFY;
req.data = reply;
req.datamax = sizeof reply;
req.status = status;
req.statusmax = sizeof status;
err = uci_exec(&req);

memset(&req, 0, sizeof req);
req.target = UCI_TARGET_DOS1;
req.command = DOS_CMD_GET_TIME;
req.args = &format;
req.arglen = 1;
req.data = reply;
req.datamax = sizeof reply;
err = uci_exec(&req);

memset(&req, 0, sizeof req);
req.target = UCI_TARGET_CONTROL;
req.command = CTRL_CMD_GET_HWINFO;
req.data = reply;
req.datamax = sizeof reply;
err = uci_exec(&req);

req.args = &sub;
req.arglen = 1;
err = uci_exec(&req);
```

OUTPUT

```
IDENTIFY: reply contains target name
GET_TIME: reply contains formatted time
GET_HWINFO: reply contains model name
```

The identify request has no arguments: zero initialization leaves `args`, `payload` and their lengths empty. The time request supplies its optional format byte. The hardware-info request is first sent without its optional selector, then with `CTRL_HWINFO_MODEL`; both forms request the model. A payload is a second independent span; set it only when required.

To discard reply data or status deliberately, leave the corresponding pointer and capacity zero:

```
req.data = NULL;
req.datamax = 0;
req.status = NULL;
req.statusmax = 0;
err = uci_exec(&req);
```

OUTPUT

```
req.datalen = 0
req.statuslen = 0
```

READ A RAW MULTI-BLOCK REPLY

```
err = uci_exec_first(&req);
while (err == ULTIMATE_OK) {
  process(req.data, req.datalen);
  if (!uci_more_blocks()) break;
  err = uci_exec_next(&req);
}
```

OUTPUT

```
Each iteration receives one reply block; the loop ends when
uci_more_blocks() = 0
```

The first call submits the request and returns one reply block. The boolean call reports whether another follows; the next call releases the current block and receives another. Do not issue an unrelated command inside this loop.

DECODE AND INSPECT RAW STATUS

```
static const uint8_t raw[] = {
  0x34,0x30,0x34,0x20,0x4e,0x4f,0x54,
  0x20,0x46,0x4f,0x55,0x4e,0x44
};
uint8_t expected;
uint16_t device_code;

expected = uci_status_format(UCI_TARGET_HTTP);
err = uci_decode_status(UCI_TARGET_HTTP, raw, sizeof raw);
device_code = uci_last_device_code();
```

`uci_status_format()` describes the target's expected encoding. `uci_decode_status()` decodes actual bytes and records their numeric status; `uci_last_device_code()` retrieves it. The decoder accepts an empty status as success, so this is also the form without status bytes:

```
err = uci_decode_status(UCI_TARGET_DOS1, NULL, 0);
```

OUTPUT

```
expected = UCI_STATUS_FMT_DECIMAL3
err = ULTIMATE_ERR_DEVICE
device_code = 404
empty DOS status = ULTIMATE_OK
```

CHAPTER 4

PROGRAMMING IN ASSEMBLY

- The register and shared-variable conventions
- Relocating the SDK's RAM and zero page
- Every service entry point by example
- Raw request blocks with optional spans

THE CALLING CONVENTION

Include `ultimate.inc` and call the unprefixed entry points. Unless an entry below says otherwise, a service returns an `ULTIMATE_*` result in `A` and may change `A`, `X`, `Y` and the flags. A byte argument uses `A`; a pointer argument uses low byte in `A` and high byte in `X`; multi-byte service arguments live in the exported `ult_*` variables.

The examples use these two ordinary ca65 macros to keep pointer and word setup visible without repeating four instructions each time:

```
    .include "ultimate.inc"

.macro setptr location, value
    lda #<value
    sta location
    lda #>value
    sta location + 1
.endmacro

.macro setword location, value
    lda #<value
    sta location
    lda #>value
    sta location + 1
.endmacro
```

OUTPUT

```
setptr ult_buf, text stores the address of text in ult_buf.
```

`setptr` and `setword` emit the same bytes; their different names make the purpose clear at each call site. Result branches always compare `A` immediately, before another instruction replaces it.

Assembly output boxes show the returned register or memory values from a representative successful run. They omit screen-printing routines so the SDK operation itself stays visible.

With the C64 character map, lowercase source letters assemble to the uppercase byte values shared by PETSCII and ASCII. Paths, filenames and HTTP text therefore appear in lowercase source below but reach the firmware in uppercase.

PLACE THE SDK'S WORKING MEMORY

The SDK's zero-page block starts at `UCI_ZP`, which defaults to `$FB`. Its size is `UCI_ZP_SIZE`. The normal variable block occupies `UCI_VARS_SIZE` bytes in BSS. Define both bases when assembling every SDK source to choose fixed locations:

```
ca65 -D UCI_VARS=$CF00 -D UCI_ZP=$A3 uci_core.s
```

OUTPUT

```
UCI variables begin at $CF00; zero-page workspace begins at $A3.
```

With `UCI_VARS` defined, the core emits no BSS; its variables become equates from that address. The SDK installs no interrupt handler and is not re-entrant, so an interrupt routine must not call it while foreground code is inside a service.

INITIALIZE, DETECT AND DESCRIBE THE MACHINE

```
    jsr ultimate_init
    cmp #ULTIMATE_OK
    bne failed

    jsr ultimate_available
    beq unavailable

    lda #<capabilities
    ldx #>capabilities
    jsr ultimate_detect
    cmp #ULTIMATE_OK
    bne failed

capabilities: .res 4
```

OUTPUT

```
A = ULTIMATE_OK
capabilities records the detected targets.
```

`ultimate_init` performs startup. `ultimate_available` returns a boolean rather than a result code. `ultimate_detect` takes the address of the four-byte capability block directly in

A/X.

Identify a target with and without the optional length output

```
    setptr ult_buf, text
    setword ult_buflen, 64
    setptr ult_outlen, text_length
    lda #UCI_TARGET_DOS1
    jsr ultimate_identify

    setword ult_outlen, 0
    lda #UCI_TARGET_DOS1
    jsr ultimate_identify

text: .res 64
text_length: .res 2
```

OUTPUT

```
text = "ULTIMATE-II DOS V1.2"
text_length = 20 with the pointer; no length is stored with zero.
```

Both calls terminate the text in `ult_buf`. The first stores its length through `ult_outlen`; zero disables that optional output.

Read the model with and without the optional length output

```
    setptr ult_buf, text
    setword ult_buflen, 64
    setptr ult_outlen, text_length
    jsr ultimate_get_model

    setword ult_outlen, 0
    jsr ultimate_get_model
```

OUTPUT

```
text = "Ultimate 64 Elite"
text_length is optional in the same way as IDENTIFY.
```

`ultimate_get_model` uses the same buffer contract but needs no target argument.

Turn a result into text

```
    lda error_code
    jsr ultimate_strerror
    sta message_ptr
    stx message_ptr + 1

error_code: .res 1
message_ptr: .res 2
```

OUTPUT

```
message_ptr points to the PETSCII text for error_code.
```

`ultimate_strerror` returns the PETSCII message pointer in `A/X`; it does not return a result code.

READ AND CHANGE THE PALETTE

```
    lda #<palette
    ldx #>palette
    jsr ultimate_palette_get

    lda #<palette
    ldx #>palette
    jsr ultimate_palette_set

    lda #6
    sta ult_color
    lda #255
    sta ult_color + 1
    lda #0
    sta ult_color + 2
    sta ult_color + 3
    jsr ultimate_palette_set_color

    jsr ultimate_palette_reset

palette: .res UCI_PALETTE_BYTES
```

OUTPUT

```
GET fills 48 bytes; SET COLOUR 6 TO RED and RESET return ULTIMATE_OK.
```

Get and set take a direct pointer in `A/X`. The single-colour entry reads index, red, green and blue from the four consecutive `ult_color` bytes. Reset restores the built-in live palette. Firmware without these commands returns `ULTIMATE_ERR_NOT_SUPPORTED`; branch on `A` before using the 48-byte result from `ultimate_palette_get`.

USE TURBO AND BADLINE CONTROL

```
    jsr ultimate_turbo_available
    beq no_turbo

    jsr ultimate_turbo_get
    sta old_speed

    lda #U64_SPEED_4MHZ
    jsr ultimate_turbo_set

    lda #0
    jsr ultimate_turbo_badlines
    lda #1
    jsr ultimate_turbo_badlines

    lda old_speed
    jsr ultimate_turbo_set

old_speed: .res 1
```

OUTPUT

```
old_speed = previous speed index
SET 4 MHZ, THEN RESTORE SPEED AND BADLINES: ULTIMATE_OK
```

The available and get calls return values directly. Set returns a result. The two badline calls show both forms explicitly: zero disables them; non-zero restores them. The final set restores the speed read at entry.

CHANGE AND READ THE CURRENT DIRECTORY

```
    setptr ult_buf, path
    jsr ultimate_chdir

    setptr ult_buf, path_buffer
    setword ult_buflen, 128
    setptr ult_outlen, path_length
    jsr ultimate_getpath

    setword ult_outlen, 0
    jsr ultimate_getpath

path: .byte "/usb1/games", 0
path_buffer: .res 128
path_length: .res 2
```

OUTPUT

```
path_buffer = "/USB1/GAMES"
path_length = 11 when the optional pointer is enabled.
```

`ultimate_chdir` consumes the terminated path through `ult_buf`. The two getpath calls show the optional length pointer enabled and disabled.

Read every directory entry

```
    jsr ultimate_opendir
    cmp #ULTIMATE_OK
    bne failed

next_entry:
    setptr ult_buf, name
    setword ult_buflen, 64
    jsr ultimate_readdir
    cmp #ULTIMATE_END
    beq directory_done
    cmp #ULTIMATE_OK
    bne failed
    lda ult_attrib
    ; use name and its DOS_ATTR_* bits here
    jmp next_entry

name: .res 64
```

OUTPUT

```
name receives one entry per call; A becomes ULTIMATE_END after the last.
```

Each readdir releases one reply block and writes attributes to `ult_attrib`. Issue no other command inside the walk. Call `uci_abort` if leaving before `ULTIMATE_END`.

OPEN, SEEK, READ, WRITE AND CLOSE FILES

```
    setptr ult_buf, read_name
    lda #DOS_FA_READ
    jsr ultimate_open
    cmp #ULTIMATE_OK
    bne failed

    lda #0
    sta ult_num
    sta ult_num + 1
    sta ult_num + 2
    sta ult_num + 3
    jsr ultimate_seek

    setptr ult_buf, file_buffer
    setword ult_buflen, 128
    setptr ult_outlen, bytes_read
    jsr ultimate_read
    jsr ultimate_close

read_name: .byte "readme.txt", 0
file_buffer: .res 128
bytes_read: .res 2
```

OUTPUT

```
file_buffer receives up to 128 bytes; bytes_read receives the actual
count.
```

Open establishes the firmware's current file. Seek selects its absolute position. Read stores bytes and, when `ult_outlen` is non-zero, their count. Close releases that same firmware-side file.

The count output is optional:

```
    ; With a readable file already open:
    setptr ult_buf, file_buffer
    setword ult_buflen, 128
    setword ult_outlen, 0
    jsr ultimate_read
```

OUTPUT

```
file_buffer receives bytes; no byte count is stored.
```

Create, write and delete a file

```
    setptr ult_buf, write_name
    lda #DOS_FA_CREATE_ALWAYS | DOS_FA_WRITE
    jsr ultimate_open
    cmp #ULTIMATE_OK
    bne failed

    setptr ult_buf, message
    setword ult_buflen, message_end - message
    jsr ultimate_write
    jsr ultimate_close

    setptr ult_buf, write_name
    jsr ultimate_delete

write_name: .byte "result.txt", 0
message: .byte "ok", 13
message_end:
```

OUTPUT

```
RESULT.TXT: 3 bytes written, closed and deleted; A = ULTIMATE_OK.
```

The open mask creates or replaces the file. Write consumes `ult_buflen` bytes from `ult_buf`; delete consumes the terminated name after the file is closed.

LOAD AND SAVE C64 MEMORY

```
    setptr ult_buf, prg_name
    setword ult_addr, 0
    jsr ultimate_load
    jsr ultimate_last_end
    sta load_end
    stx load_end + 1

    setword ult_addr, $C000
    jsr ultimate_load

    setptr ult_buf, raw_name
    setword ult_addr, $C000
    setword ult_max, 2048
    jsr ultimate_bload

    setptr ult_buf, save_name
    setword ult_addr, $0400
    setword ult_max, 1000
    jsr ultimate_save

prg_name: .byte "game.prg", 0
raw_name: .byte "font.bin", 0
save_name: .byte "screen.bin", 0
load_end: .res 2
```

OUTPUT

```
load_end = address after loaded PRG data
FONT.BIN loads at $C000; SCREEN.BIN receives 1000 bytes.
```

An `ult_addr` of zero makes load use the PRG header; `$C000` shows the explicit-address form. `ultimate_last_end` returns the previous load end in `A/X`. Bload consumes no header and obeys `ult_max`; save uses `ult_max` as its length.

MOVE BYTES THROUGH THE REU

```
    jsr ultimate_reu_available
    beq no_reu
    jsr ultimate_reu_size
    sta reu_banks
    stx reu_banks + 1

    setword ult_addr, $0400
    lda #0
    sta ult_reu
    sta ult_reu + 1
    sta ult_reu + 2
    sta ult_reu + 3
    setword ult_reulen, 1000
    lda #0
    sta ult_reulen + 2
    sta ult_reulen + 3
    jsr ultimate_reu_stash
    jsr ultimate_reu_fetch

reu_banks: .res 2
```

OUTPUT

```
reu_banks = installed REU size in 64K banks
1000 bytes stashed and fetched; A = ULTIMATE_OK.
```

Size returns 64K banks in A/X. Stash copies C64 RAM to the REU; fetch copies it back. Both read the C64 address, 32-bit REU address and 32-bit length from the shared variables.

Move the current file directly to and from the REU

Both transfers below use REU address zero and a 65536-byte length:

```
    lda #0
    sta ult_reu
    sta ult_reu + 1
    sta ult_reu + 2
    sta ult_reu + 3
    sta ult_reulen
    sta ult_reulen + 1
    lda #1
    sta ult_reulen + 2
    lda #0
    sta ult_reulen + 3
```

OUTPUT

```
ult_reu = $00000000; ult_reulen = 65536.
```

File to REU

```
    ; Open ASSETS.BIN first; it becomes the current DOS file.
    setptr ult_buf, asset_name
    lda #DOS_FA_READ
    jsr ultimate_open
    cmp #ULTIMATE_OK
    bne failed
    lda #0
    sta ult_num
    sta ult_num + 1
    sta ult_num + 2
    sta ult_num + 3
    jsr ultimate_seek
    jsr ultimate_reu_load
    jsr ultimate_close

asset_name: .byte "assets.bin", 0
```

OUTPUT

```
ASSETS.BIN to REU $000000: 65536 bytes
```

REU to file

```
    ; Open a writable current file before the transfer.
    setptr ult_buf, capture_name
    lda #DOS_FA_CREATE_ALWAYS | DOS_FA_WRITE
    jsr ultimate_open
    cmp #ULTIMATE_OK
    bne failed
    jsr ultimate_reu_save
    jsr ultimate_close

capture_name: .byte "capture.bin", 0
```

OUTPUT

```
REU $000000 to CAPTURE.BIN: 65536 bytes
```

The REU address is zero and the length bytes encode 65536. Load and save take no filename: each operates on the firmware's current open file established by the preceding `ultimate_open`.

INSPECT NETWORK INTERFACES

```
    jsr ultimate_net_ifcount
    cmp #ULTIMATE_OK
    bne failed
    lda ult_iface
    beq no_interfaces

    lda #0
    sta ult_iface
    setptr ult_buf, mac
    jsr ultimate_net_macaddr
    setptr ult_buf, ipconfig
    jsr ultimate_net_ipconfig

mac: .res 6
ipconfig: .res 12
```

OUTPUT

```
ult_iface = interface count; mac receives 6 bytes; ipconfig receives
12.
```

The count is returned in `ult_iface`; the address calls take an interface index there and place fixed-size replies at `ult_buf`.

USE TCP AND UDP SOCKETS

```
    setptr ult_buf, host
    setword ult_port, 6502
    jsr ultimate_net_connect
    cmp #ULTIMATE_OK
    bne failed

    setptr ult_buf, message
    setword ult_buflen, message_end - message
    jsr ultimate_net_write
    ; ult_socklen is now the number accepted.

host: .byte "192.168.1.50", 0
message: .byte "hi", 13
message_end:
```

OUTPUT

```
TCP handle is in ult_sock; ult_socklen = 3 bytes accepted.
```

Read and close the TCP socket

```
poll:
    setptr ult_buf, socket_buffer
    setword ult_socklen, 128
    jsr ultimate_net_read
    cmp #ULTIMATE_END
    beq peer_closed
    cmp #ULTIMATE_OK
    bne close_socket
    lda ult_socklen
    ora ult_socklen + 1
    beq poll

close_socket:
    jsr ultimate_net_close

socket_buffer: .res 128
```

OUTPUT

```
Reply bytes are in socket_buffer; close returns ULTIMATE_OK.
```

UDP

```
    setptr ult_buf, host
    setword ult_port, 6502
    jsr ultimate_net_udp
    cmp #ULTIMATE_OK
    bne failed
    jsr ultimate_net_close
```

OUTPUT

```
UDP handle opens and closes; A = ULTIMATE_OK.
```

Connect and UDP return the handle in `ult_sock`. Write takes its length from `ult_buflen` and reports accepted bytes in `ult_socklen`. Read takes capacity and returns stored bytes through `ult_socklen`. A zero-byte success means poll again. `ULTIMATE_END` means the handle is already gone; do not close that ended handle.

USE THE HTTP CLIENT

```
    setptr ult_url, url
    setptr ult_buf, http_reply
    setword ult_buflen, 512
    jsr ultimate_http_get
    ; ult_httplen is the number stored.

url: .byte "http://192.168.1.50/", 0
accept_header: .byte "accept: text/plain", 0
http_reply: .res 512
body_handle: .res 1
```

OUTPUT

```
One-call GET stores response bytes; A = ULTIMATE_OK.
```

Multi-step GET without a body

```
    setptr ult_url, url
    lda #HTTP_VERB_GET
    jsr ultimate_http_open
    cmp #ULTIMATE_OK
    bne failed

    setptr ult_url, accept_header
    jsr ultimate_http_header
    lda #HTTP_BODY_NONE
    sta ult_body
    setptr ult_buf, http_reply
    setword ult_buflen, 512
    jsr ultimate_http_exchange
    jsr ultimate_http_close
```

OUTPUT

```
GET without a body stores response bytes; close returns ULTIMATE_OK.
```

POST with a body handle

```
    setptr ult_url, url
    lda #HTTP_VERB_POST
    jsr ultimate_http_open
    cmp #ULTIMATE_OK
    bne failed
    lda body_handle
    sta ult_body
    jsr ultimate_http_exchange
    jsr ultimate_http_close

    jsr ultimate_http_free_all
```

OUTPUT

```
POST stores response bytes; close and free-all return ULTIMATE_OK.
```

The one-call GET creates and releases its own slot. The multi-step form stores the request handle in `ult_http`; header, exchange and close use it there. The two exchange calls show the no-body and body-handle forms explicitly. `ultimate_http_free_all` recovers slots abandoned by an earlier program.

PROBE AND CONFIGURE THE RAW TRANSPORT

```
    jsr uci_present
    beq no_interface
    jsr uci_ident
    sta raw_ident

    jsr uci_get_timeout_a
    pha
    lda #UCI_TIMEOUT_FOREVER
    jsr uci_set_timeout_a
    jsr run_slow_command
    tax
    pla
    jsr uci_set_timeout_a
    txa

    jsr uci_last_code
    sta device_code
    stx device_code + 1
    jsr uci_abort
```

OUTPUT

```
uci_present returns nonzero; raw_ident receives the interface ID.
device_code receives the last target status; abort returns ULTIMATE_OK.
```

Present and ident are direct register reads. Timeout zero waits forever; this pattern restores the previous budget while preserving the command result. Last code returns the raw target status in A/X. Abort releases an unfinished exchange and is safe while idle.

RAW COMMANDS: OPTIONAL ARGUMENTS

```
    jsr uci_req_clear
    lda #UCI_TARGET_DOS1
    sta uci_req + UCI_REQ_TARGET
    lda #UCI_CMD_IDENTIFY
    sta uci_req + UCI_REQ_COMMAND
    setptr uci_req + UCI_REQ_DATA, raw_reply
    setword uci_req + UCI_REQ_DATAMAX, 64
    jsr uci_exec_block

    jsr uci_req_clear
    lda #UCI_TARGET_DOS1
    sta uci_req + UCI_REQ_TARGET
    lda #DOS_CMD_GET_TIME
    sta uci_req + UCI_REQ_COMMAND
    setptr uci_req + UCI_REQ_ARGS, time_format
    setword uci_req + UCI_REQ_ARGLEN, 1
    jsr uci_exec_block

    jsr uci_req_clear
    lda #UCI_TARGET_CONTROL
    sta uci_req + UCI_REQ_TARGET
    lda #CTRL_CMD_GET_HWINFO
    sta uci_req + UCI_REQ_COMMAND
    setptr uci_req + UCI_REQ_DATA, raw_reply
    setword uci_req + UCI_REQ_DATAMAX, 64
    jsr uci_exec_block

    setptr uci_req + UCI_REQ_ARGS, hwinfo_model
    setword uci_req + UCI_REQ_ARGLEN, 1
    jsr uci_exec_block

time_format: .byte 0
hwinfo_model: .byte CTRL_HWINFO_MODEL
raw_reply: .res 64
```

OUTPUT

```
IDENTIFY returns the target name.
GET_TIME returns the formatted time.
Both GET_HWINFO forms return the model name.
```

Clearing the request makes every optional pointer and length zero. The first command

therefore has no arguments, payload or status buffer. The second adds the optional GET_TIME format byte. The hardware-info command is then sent without its selector and with an explicit model selector. Set payload, data or status spans in the same pointer-plus-length form only when needed.

Use a caller-owned request block

```
    lda #<my_request
    ldx #>my_request
    jsr uci_exec

my_request: .res UCI_REQ_SIZE
```

OUTPUT

```
A = request result; lengths and retained bytes are written into
my_request.
```

`uci_exec` takes the request pointer directly. Use it instead of the global block when the program owns more than one request.

READ A RAW MULTI-BLOCK REPLY

```
    lda #<my_request
    ldx #>my_request
    jsr uci_exec_first
    cmp #ULTIMATE_OK
    bne failed

next_block:
    ; use this block's UCI_REQ_DATALEN bytes
    lda uci_more
    beq all_blocks
    jsr uci_exec_next
    cmp #ULTIMATE_OK
    beq next_block
```

OUTPUT

```
Each iteration receives one block; uci_more = 0 after the final block.
```

First submits the request and returns one block. `uci_more` is the boolean state byte that says whether another follows; next releases the current block and receives it. Issue no unrelated command until the final block or call `uci_abort` when stopping early.

CHAPTER 5

OTHER TOOLCHAINS

- Building and identifying the blob
- The stable jump table
- The parameter block
- Calling it from BASIC
- Choosing a different base address

BUILD AND IDENTIFY THE BLOB

`make blob` produces `bindings/blob/build/ultimate-8000.bin`. Load it at `$8000`. The first three bytes are the PETSCII signature `UCI`; the fourth is the blob ABI version.

This example demonstrates checking the default build before calling it:

```
    lda $8000
    cmp #$D5    ; PETSCII 'U'
    bne no_sdk
    lda $8001
    cmp #$C3    ; PETSCII 'C'
    bne no_sdk
    lda $8002
    cmp #$C9    ; PETSCII 'I'
    bne no_sdk
    lda $8003
    cmp #1      ; ABI version
    bne wrong_version
```

OUTPUT

```
All four comparisons match; execution continues with ABI version 1.
```

CALL THE STABLE JUMP TABLE

Entries begin at base plus `$04`. Each entry is a three-byte `jmp`; new entries are appended so published offsets do not move.

```
    jsr $8004    ; +$04 uci_init
    jsr $801C    ; +$1C ultimate_init
```

OUTPUT

```
A = ULTIMATE_OK after each initialization call.
```

This demonstrates the two initialization entries. Use `ultimate_init` for application code; the lower-level `uci_init` entry is exposed for transport users. Both return a result in `A`.

The complete offset table is in `bindings/blob/README.md`.

USE THE PARAMETER BLOCK

Service entries that need several arguments use a 512-byte block at base plus `$100`. These are the fields needed by the example below:

Offset	Field	Purpose
`+$00`	`bp_result`	last service result
`+$05`	`bp_len`	length in or out
`+$29`	`bp_name`	40-byte area for a terminated name
`+$51`	`bp_reply`	256-byte reply area

Later fields extend beyond the first page while retaining their published offsets. Consult the blob README for the complete layout rather than deriving an address from this subset.

CALL THE BLOB FROM BASIC

The page-aligned block also gives BASIC a calling convention it can express with `POKE`, `PEEK` and `SYS`:

```
10 B = 32768 : P = B + 256
20 IF PEEK(B)<>213 OR PEEK(B+1)<>195 THEN PRINT "NO SDK":END
25 IF PEEK(B+2)<>201 THEN PRINT "NO SDK":END
30 SYS B + 28 : IF PEEK(P) THEN PRINT "INIT FAILED": END
40 SYS B + 175
50 PRINT "REU BANKS:"; PEEK(P+5) + PEEK(P+6)*256
```

Line 30 calls `ultimate_init` at offset `$1C` and reads `bp_result`. Line 40 calls `reu_size` at offset `$AF`; that entry returns the 16-bit bank count in `bp_len`. This demonstrates using the blob without installing the BASIC wedge.

CHOOSE A DIFFERENT BASE ADDRESS

Prefer building at the final base: `make -C bindings/blob BASE=7000` links a `$7000` image directly. When the address is known only at run time, the build also emits a relocation table for `reloc.s`. The relocated emulator suite moves the default blob, erases the original and calls the moved copy.

CHAPTER 6

APPENDIX

- Result codes

RESULT CODES

0	ULTIMATE_OK	success
1	ULTIMATE_ERR_NO_DEVICE	command-interface signature absent
2	ULTIMATE_ERR_TIMEOUT	polling budget exhausted
3	ULTIMATE_ERR_PROTOCOL	malformed or inconsistent transport reply
4	ULTIMATE_ERR_NOT_SUPPORTED	target, command or direct hardware unavailable
5	ULTIMATE_ERR_INVALID_ARGUMENT	caller supplied an invalid argument
6	ULTIMATE_ERR_IO	filesystem, network or transfer failure
7	ULTIMATE_ERR_DEVICE	target reported a failure
8	ULTIMATE_ERR_TRUNCATED	retained buffer did not hold the whole reply
9	ULTIMATE_ERR_ABORTED	exchange was abandoned
10	ULTIMATE_END	directory or socket stream ended normally

`ultimate_strerror()` returns a printable description. `ultimate_device_code()` returns the last target's raw numeric status for diagnostics; its meaning depends on the target.

ONE LIBRARY, FOUR WAYS IN

Use files, REU memory, the live palette, sockets and HTTP from a C64 program without implementing the command-interface handshake.

BASIC, cc65 C, ca65 assembly and the standalone jump table all reach the same 6502 implementation.

- BASIC V2 keywords at the `READY.` prompt
- A cc65 library and public headers
- A documented ca65 calling convention
- A standalone jump table for other toolchains

```
READY.
PRINT UREU
 32
READY.
```

This expression reads the enabled REU size in 64K banks; 32 means 2MB.

FOR THE 1541 ULTIMATE-II, THE ULTIMATE 64 AND THE COMMODORE 64 ULTIMATE

FIRST EDITION · MIT LICENCE